

01 SNS

AWS Simple Notification Service (SNS)

SNS Core Concepts

AWS SNS (Simple Notification Service) is a **push model based** or **broadcast model based notification service**. It is designed for scenarios where a single message needs to be **broadcasted to different subscribers** using the **fan-out messaging pattern**.

SNS vs. Kafka: SNS should not be considered like Kafka or a managed Kafka service. While both are messaging services, SNS operates on a push model, meaning messages are automatically delivered to subscribers, whereas Kafka typically requires consumers to poll for messages. Additionally, while Kafka excels at high-throughput streaming and log aggregation, SNS is optimized for simple pub/sub notifications and seamless integrations within the AWS ecosystem.

SNS Architecture and Use Cases

SNS works by allowing users to create **topics** to which messages are published. Subscribers then receive these messages or notifications on those topics. SNS is a **regional service**, meaning topics are created within specific AWS regions.

Common Use Cases:

- **Broadcasting notifications (Fan-out pattern):** Send the same message to various downstream subscribers simultaneously.
- **Support team acknowledgements:** Receive notifications, e.g., when a file is uploaded to an S3 bucket, to track receipt or processing status.
- **Event-driven workflows:** Integrate with other services where a single event needs to trigger multiple processes (e.g., S3 upload triggering a Lambda to process data and SNS to notify other workflows like Spark jobs or Step Functions).
- **Alerting and Monitoring:** Generate alerts based on success or failure of processes.

Typical SNS Subscribers:

- **Mobile numbers (SMS)**
- **Email addresses**
- **Other AWS services like:**
 - **SQS queues**
 - **Lambda functions**
 - **HTTP/S endpoints**
 - **Application endpoints** for mobile devices

SNS Topic Types: Standard vs. FIFO

When creating an SNS topic, you can choose between two types:

1. Standard Topic:

- **No message ordering guarantees:** Messages may arrive out of order to subscribers
- **At least once message delivery:** The same message might be delivered more than once. Downstream systems need to handle de-duplication
- **Highest publish throughput:** Offers higher throughput compared to FIFO
- **Multiple subscription protocols:** Supports HTTP/S, email, SMS, Lambda, SQS, and more. This is the one to use if you need flexible delivery options

2. FIFO (First-In-First-Out) Topic:

- **Strictly preserved message ordering:** Messages are delivered in the exact order they are published
- **Exactly once message delivery:** Guarantees a message is delivered exactly once
- **Lower throughput:** Up to **3,000 publishes per second** when using batching, or 300 publishes per second without batching
- **Limited subscription protocol:** Only **SQS FIFO queues** can be used to deliver messages from an SNS FIFO topic

SNS Key Features

- **Pub/Sub Messaging:** Follows a publish-subscribe paradigm using topics
- **Multiple Protocols:** Supports various communication protocols for delivering messages
- **Fan-out Architecture:** A single message published to an SNS topic can be delivered to many recipients across different supported protocols
- **Durability:** Messages are stored redundantly across multiple servers and data centers, providing high availability and durability
- **Content Filtering:** Allows filtering messages delivered to each subscription, ensuring subscribers receive only messages of interest. You can define filter policies based on message patterns or JSON content
- **Message Attributes:** Supports message attributes for additional metadata and enhanced filtering capabilities
- **Access Control:** Integrates with **AWS IAM** for granular access control, allowing specification of who can publish or subscribe to a topic. You can use **ARNs** in policies to restrict publishing access to specific services like S3 or Lambda
- **Large Message Size:** For messages exceeding the normal size limit, SNS can store them in an **Amazon S3 bucket** and send a pointer to the message in the notification, rather than the entire body
- **Dead Letter Queues (DLQs):** Supports Dead Letter Queues for handling failed message deliveries, ensuring reliable message processing
- **Monitoring:** Integrated with **CloudWatch** for health monitoring and logging
- **Encryption:** Supports encryption in transit (using HTTPS) and at rest (using AWS KMS)
- **Regional Service:** Topics are created within specific AWS regions
- **Costing:** Users pay for the number of requests, messages delivered, and data transferred